

VICINIA · QA RUNBOOK

Full Order Flow Validation Guide

A step-by-step script for manually exercising the complete customer → merchant → delivery pipeline against the local dev stack — browsing, cart, checkout, fulfillment, live delivery assignment, and the notification trail at every hop.

Vicinia — hyperlocal multi-merchant commerce platform

Local stack: `http://localhost:5173` (frontend) · `http://localhost:8080` (api-gateway)

Full Order Flow Validation Guide

0. Prerequisites

- ☐ Infra containers running — `./start-infra.sh` from the repo root (Postgres, Mongo, Redis, Redpanda, etc.)
- ☐ All backend services up and registered in Eureka
- ☐ Frontend dev server running — `npm run dev` in `frontend/customer-app`, served at `http://localhost:5173`
- ☐ Four browser sessions available (separate browsers, profiles, or incognito windows) — one per role, since each holds its own login session

1. Test accounts

Same login page for every role — `http://localhost:5173/login` — the app routes you by the account's role after login.

Role	Email	Password	Notes
CUSTOMER	<code>customer1@vicinia.test</code>	<code>12345678</code>	Has a default address in Bangalore
MERCHANT	<code>merchant1@vicinia.test</code>	<code>12345678</code>	Owns Green Valley Grocers (LIVE)
MERCHANT	<code>merchant2@vicinia.test</code>	<code>12345678</code>	Owns City Fresh Mart (LIVE) — recommended for testing, see §2
DELIVERY	<code>rider1@vicinia.test</code>	<code>12345678</code>	Delivery partner
DELIVERY	<code>rider2@vicinia.test</code>	<code>12345678</code>	Delivery partner
ADMIN	<code>admin@vicinia.test</code>	<code>12345678</code>	Platform admin

2. Align store & rider location *before* you start

Delivery assignment is **real geo-matching**, not a mock. When a merchant marks an order "ready," delivery-service runs a Redis GEO radius search — nearest online rider within **15 km** (`vicinia.delivery.search-radius-km`, default 15) of the **merchant's stored latitude/longitude**. If nobody online is in range, the task silently sits as `PENDING_ASSIGNMENT` — no error anywhere — and a background reaper only retries it every 30s.

KNOWN ISSUE IN SEED DATA

The two seeded LIVE stores have inconsistent test coordinates: **City Fresh Mart** sits at 12.9352, 77.6146 (central Bangalore), while **Green Valley Grocers** sits at 22.77, 86.16 — actually in Odisha, ~1,500 km away. An order from Green Valley Grocers will never get assigned to a real local rider until its coordinates are fixed.

1. Log in as `merchant2@vicinia.test` → **Merchant** → **Store settings**.
2. Under "Location," click **Use my current location** — do this from the same machine/browser you'll use for the rider in step 4 below, so the coordinates actually match.
3. Click **Save store profile**.
4. Plan to place your test order against **City Fresh Mart** specifically (not Green Valley Grocers, unless you repeat this fix for it too).

HEADLESS / CI / NO REAL GPS

Spoof geolocation instead of relying on real GPS: Chrome DevTools → **Sensors** → **Location**, or in a Playwright script, `context.setGeolocation({ latitude, longitude })`. Use the *same* coordinates for the merchant's "use my location" click and the rider's "go online" click in step 4.

3. Customer places an order

CUSTOMER `customer1@vicinia.test`

1. Log in at `http://localhost:5173/login`.
2. On the home page, under "Shops near you," click the **City Fresh Mart** card — this opens its store page (`/store/:merchantId`) with its real catalog.
3. Click **Add** on 1–2 items — the cart badge in the navbar increments.
4. Go to **Cart** — check thumbnails, quantities, and the "Clear cart" button render correctly, then click **Proceed to checkout**.
5. Pick or add a delivery address, confirm payment (test flow — no real Razorpay charge is made), and place the order.
6. Note the **order ID** shown on the confirmation / order-detail page — you'll need it to spot-check later.

4. Merchant accepts and prepares it

MERCHANT

`merchant2@vicinia.test`

1. Log in (separate browser/session from the customer).
2. Go to **Merchant** → **Orders**. The new order should appear in the pending queue, and you should get a "New order received" notification in the bell dropdown.
3. Click **Accept**.
4. Once prepared, click **Mark ready** — this is the moment that publishes `order.ready` and triggers the real-time delivery assignment search described in §2.

5. Delivery partner picks up and delivers

DELIVERY

`rider1@vicinia.test`

1. Log in (same machine/browser you used for the store's "use my location" in §2, or the same spoofed coordinates).
2. Click **Go online** — grant browser location access when prompted. This registers you in Redis GEO as an available rider.
3. Within a few seconds of the merchant marking the order ready, you should receive a "New pickup assigned" notification. Clicking it deep-links straight into the active task with the order pre-loaded (no need to type the order ID manually).
4. Step through the task: **Accept** → **Mark picked up** → **Mark delivered**.

NOTHING ARRIVING?

If no assignment notification shows up after ~30–60s, the rider is most likely outside the merchant's 15km search radius (see §2), or the merchant never actually clicked "Mark ready." Confirm both before assuming something is broken.

6. Customer confirms delivery

CUSTOMER

`customer1@vicinia.test`

1. Go to **Orders** → **this order** and confirm the status reads "Delivered."
2. Check the notification bell — you should have an "Order confirmed" notification from earlier in the flow, and the order's status should have tracked live through each stage.

7. Admin spot-check (optional)

ADMIN

`admin@vicinia.test`

1. Log in.
2. **Admin** → **Merchants** — confirm the "All merchants" tab shows persistent history, not just the pending queue.
3. **Admin** → **Settlements** / **Products** / **Coupons** as needed for broader coverage.

8. What this exercises end-to-end

This script walks the full chain the backend's Kafka event pipeline is built around:

Stage	Event	Consumers notified
Order placed	<code>order.confirmed</code>	Customer + merchant notifications
Merchant accepts	<code>order.accepted</code>	Order status update
Merchant marks ready	<code>order.ready</code>	Delivery-service (triggers Redis GEO assignment search)
Rider assigned	<code>delivery.assigned</code>	Rider "New pickup assigned" notification
Rider delivers	<code>delivery.delivered</code>	Order marked complete

9. Troubleshooting quick reference

Symptom	Likely cause	Fix
Login hangs / never resolves	Docker infra containers silently exited	<code>docker ps -a</code> — if vicinia-prefixed containers show "Exited," run <code>./stop-infra.sh</code> then <code>./start-infra.sh</code>
Delivery task stuck at "pending"	No online rider within 15km of the merchant's coordinates	Re-align locations per §2, or increase <code>vicinia.delivery.search-radius-km</code> for local testing
Cross-store cart conflict banner	Cart already has items from a different store — a Vicinia order is single-merchant only	Expected behavior — use "Clear cart & add this instead," or manually clear the cart first
Notification bell not updating	Client polls every 30s, not real-time push	Wait up to 30s, or manually refresh